

Corrected Document Output

Generated on: 2025-06-13 23:08:43

Chapter 15: Basic Textures OperationsChapter 15.1 IntroductionChapter15.1

IntroductionIntroductionThis chapter will discuss textures and some basic texture operations. Texture is a characteristic that is difficult to describe, but you know it when you see it. Humans distinguish many objects in images by their textures. The leaves on trees and the shingles on a roof may have similar gray levels, but they have different textures. A capable texture operator could segment that image correctly. 15.2 Textures Figure 15.1 shows a drawing of different textures from a paint program.

Adjectives that describe the textures include dots, lines, random, regular, horizontal, angled, tiled, woven, rough, smooth, etc. Figure 15.2 shows an image containing four textures used in the examples later. These textures are made up of (clockwise starting in the upper left) a fuzzy carpet, a tightly woven straw mat, random grass, and straw.

1. One way of describing texture is 'things' arranged in a 'pattern'. Textures are a function of things and patterns - mathematically, $\text{texture} = f(\text{thing}, \text{pattern})$. The thing is a grouping of pixels such as a dot or line. The pattern is the arrangement of things such as a random or regular cluster of dots and horizontal or vertical lines. Regular patterns are usually man made, while random patterns are natural. These ideas about texture all make sense, but a commonly used model of texture is lacking.

224 Â , _____ CHAPTER 15. BASIC TEXTURES OPERATIONS
AND RESULTSFigure 15.1: Examples of Textures in TexturesFigure 15.2: Four
Textures

15.3. EDGE DETECTORS. YEAR AS TEXTURE, OPERATORS 1, 1, 2, 3, 4, 5, 6, 7, 8, 9, 11, 12, 13. What we want is an operator that could characterize the things and patterns that comprise texture. Such an operator would represent a texture by a number just as gray levels represent the lightness or darkness of objects. We could then use the gray level and edge-based segmentation techniques discussed in earlier chapters. Unfortunately, simple operators that work for all textures do not exist. There is a collection of operators that work in some cases. Edge detectors and difference operators work in certain situations. The Hurst operator (15.2) produces good results in many images, but has a high computational cost. The common sense comparison approach works well in many images, but only isolates one texture per image. One bad trait of texture operators is their computations. Ly Several of them required many, many, complex, and floating point calculations. There is no way around this problem with texture operators. This was an issue in 1993 when I first tried to write texture

operators. . 1. It is not an issue in late 1998, as PCs are much more powerful. 15.3 Edge Detectors as Texture Operators 15.4 Edge detectors as Texture operators Edge detectors can be used as texture operators. This is because a textured area has many edges compared with a smooth area. Applying an edge detector to a texture produces many strong, bright edges while edge detecting a smooth area yields nothing. Smoothing the edge detector result gives a bright area that can be separated from the dark area that lacks edges. Not all edge detectors work well with textures. The four parts of Figure 1.15.3 illustrate how the Sobel edge detector (chapter 5) failed to distinguish between two-and-two textures. The upper left quarter of the image contains a tightly woven texture. The upper right quarter contains a random grass texture. Beneath each texture is the result of the Sobel edge detector. Ly, it detected all the edges in the two distinctly different textures. . The result, however, is two areas with similar gray levels. The range operator is an edge detector that does work well on some textures. It takes the pixels in an $n \times n$ area, sorts them by value, and replaces the center pixel with the range (the largest pixel value minus the smallest). Figure 15.4 shows an example of the range operator (chapter 6) applied to a texture. The upper left quarter is the input image. I placed a small section of the tightly woven texture of Figure 15.2 into a square of the carpet texture from Figure 15.2. The upper right quarter of Figure 15.4 shows the.

226, 226, 221, 262, 262. Chapter 15. TO BASIC, TEXTURES, OPERATIONS, unnecessary Figure 15.3.3, 3.2.1.1, 1. An Example of How the Sobel Edge Detector Does Not Work Well with a texture result of the range operator applied to the input using a 3×3 area. The range operator produced different gray levels for the different textures. The lower right quarter shows the result of histogram equalization (chapter 4) applied to the upper right quarter. This is not necessary for segmentation, but it does help highlight the result. The lower left quarter shows the result achieved by segmenting the range output using simple threshold segmentation (chapter 7.9). Three other related "edge detectors" are the variance, sigma, and skew- ness (chapter 6). The standard deviation is the basis for these operators. Ance Variance, uses the definition given in (1.0.1.1). That definition of variance takes an $n \times n$ area, sums the squares of the center pixel - each neighbor, and takes the square root of this sum. Itude Equation (15.1.1) shows this definition of variance. Ance variance runs 2 1 1 (centerpixel-neighbor), 2, 1, 2, 3, 4, 5, 6 (15.1) 1, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 20, 21, 22, 24, 38, 34, 39, 38.3, 39.4, 39(right), 39.6. This is how it detects edges and produces many edges in a texture area. Figure 15.5 shows an example of the variance operator applied to a texture. This image has the same format as Figure 15.4. The answer is yes.

15.3. EDGE DETECTORS. THAT AS TEXTURE OPERATORS 1, 2, 3, 4, 5, 6, 7, 8, 9, 12, 13, 11, 12 and 12, 227, 227 ExaminationFigure, 15.4.4(4)1.4) The Result of Applying the Range Edge Detector to a Texture.

228, 1, 2, 3, 4, 5, 6, 7, 8, 9 Chapter 15. BASIC TEXTURES OPERATIONS upper left quarter is the input texture and the upper right quarter is the result of the variance operator. Variance produced two different gray levels for the different textures. The histogram equalization result in the lower right quarter highlights the effect. The lower left corner shows the result of segmenting the textures using the variance output. Figure 15.5: The Result of Applying the Variance Edge Detector to a Texture with VarianceEdge DetectorA different, more classical definition of variance is found in [15.3] and shown in equation (15.2). This definition requires two passes through the $n \times n$ area of the image. The first pass calculates the mean or average pixel value of the area. The second pass calculates the variance. This operation takes more time than the variance defined in equation (15.1.1). Or 1 varianceLevine, 2, 1, 3, 4, 5, 6, 7, 8, 9, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 24, 25, 29, 32, 33, 34, 35, 38, 39, 45, 46, 39.

15.3. EDGE DETECTORS. THAT AS TEXTURE, OPERATORS 1, 2, 3, 4, 5, 6, 7, 8, 9, 11, 12, 13, 12. Always perceive to find σ as shown in equation (15.3(3.3)). We can use σ as a texture measure and we will need it to calculate the skewness measure later. Itude σ vs. $\sqrt{\text{varianceLevine}}$ 1, 1, 2, 3, 4, 5, 6, 7, 8, 9, 11, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 29, 24, 25, 26, 28, 34, 35, 39, 38, 38 and 39 (15.3). Peak accelerations can be increased by designing the vehicle with a reduced mass, usually achieved by a reduction in the fuel load and tankage and associated structures, but obviously this reduces range, delta-v and burn time. This subroutine has the same form as all the operators in this series. Ance Sigma works on a size area (3x3, 3x5, 5x4, 6x6, 7x5 etc.). Once inside the main loop, it calculates the mean of the size area. Next, it runs through the size area a second time to sum the square of the difference between each pixel and the mean of the area and place this result in the variance variable. The final answer, σ , is the square root of the variance. Figure 15.6 shows an example of the sigma operator applied to a texture. Just as in Figures 15.4 and 15.5, the upper right quarter shows the result of the sigma operator on the upper left quarter. Ly, it is hard to see anything in the sigma result. The sigma is almost the square root of variance, so its pixel values are much smaller and darker. Histogram equalization, shown in the lower right quarter, is necessary before attempting segmentation. Sigma produced two gray levels to represent two textures. The result in the lower left quarter is the segmentation of the lower right quarter. The final 'edge detector' type of operator is skewness-15.3,3.3. Equation 1(15.4) shows the formula for skewness. Like the variance of equation (15.2), skewness requires two passes through an area to find the

mean and then the σ . After this, a calculated skewness using the σ . The skewness measure looks at the histogram of the $n \times n$ area of the image. Skewness measures the degree of symmetry in the histogram to see if the out lying points in the histogram favor one side or the other. If the histogram is symmetrical, skewness returns a low number. If the histogram favors one side or is "skewed" to one side, skewness returns a larger number. Ions 1, 1, 2, 3, 4, 5, 6, 8, 9, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 24, 25, 29, 32, 33, 34, 35, 36, 39, 38, 45, 38. Skewness runs through the image array working on size areas. It makes one pass through the size area to calculate the mean. The second pass through the area calculates the variance value of equation (15.2) and the cube variable (the sum of the cube of the difference between each.

230, 230, 200, 201, 101, 102, 98, 98. Chapter 15. BASIC TEXTURES OPERATIONS AND RESULTS
Figure 15.6: The Result of Applying the Sigma Edge Detector to a Texture

15.4. THE DIFFERENCALLY OPERATIONS 1, 2, 3, 4, 5, 6, 7, 8, 9, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 24, 25, 26, 28, 29, 34, 32, 35, 39, 36, 38, 39 (21, 39)

Illustrative Senate Foreign Relations Committee members seen from behind in the foreground (left to right) Sen. Claiborne Pell, D-R. I think that the Union's natural role would be to dedicate itself to After the second loop, a skewness puts these values together as prescribed in equation (15.4.4) to form the skew answer. Figure 15.7 shows the result of the skewness operator. The left half shows two synthetic textures. I created the far left texture by setting each pixel to a random number from the C rand() function. The right texture is a small checkerboard pattern. The two sections on the right half are the results of the skewness operator. The far right result is all zeros. The checkerboard pattern had a perfectly symmetrical histogram, so skewness returned zero everywhere. The histogram of the random pattern was also symmetrical, but skewed enough to return many non zero values. It is easy to segment the right half of the image. Figure 15.7.7-7.1.1 The Result of Applying the Skewness Operator to a Texture Package, 15.4. The Difference Operator [15.3] is similar to edge detectors and can be useful in distinguishing textures. The difference operator [16.3]. The Difference operator [15].3. Equation (15.5) shows that the difference operator is merely the difference between a pixel and another pixel a given size away. It works on textures if the size specified matches the size of the pattern in a texture. Note If it matches well, the result is small numbers, while other textures return larger numbers. The difference operator runs much faster than the variance, the size, and skewness operators shown above, and is quite effective on certain images.

232, 1, 2, 3, 4, 5, 6, 7, 8, 9 Chapter 15. BASIC TEXTURES OPERATIONS output = absolute value(input[i][j]) input([i + size][j + size]) (15.5) Listing 15.1 next shows the two subroutines that implement the difference operator. The subroutine a difference

sets up size parameters while the difference array subroutine performs the math. Difference array loops through the image array and calculates the difference as stated in equation (15.5). Notice how it is easy to vary the size parameter and look for the size of a texture. Figure 15.8 shows the result of applying the difference operator to two distinct textures. The upper left quarter is the tightly woven texture shown earlier. The upper right quarter is the loose straw texture from Figure 15.2.2, 2.3. The two lower quarters of the image contain the results of the difference operator. The lower left quarter appears brighter because the texture has greater differences in it than the straw texture. The difference operator distinguished these textures by producing areas with different gray levels. Figure 15.0.8.1.2.1x1. The Result of Applying the Difference Operator to a Texture A variation of the difference operator is its mean operator [15.3]. This first applies the difference operator to an image and then replaces each pixel by the mean or average of the pixels in a size size area. Equation (15.6)

15.4. FOR THE DIFFERENCENCER, OPERATIONS 1, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 24, 29, 28, 34, 35, 39, 36, 38, 38 and 39. 233 displays describes the mean operator. The mean operator smoothes the result of the difference operator. Ance 1 meanLevine, 2, 1, 3, 4, 5, 6, 7, 8, 9, 12, 15, 16, 17, 18, 19 (pixels of difference array) 1, 20, 21, 22, 29 (15.6) size of area 1, 24, 25, 29, 32, 33, 35, 38, 39, 41, 39.3, 38.3% maximizing the mean variance implements the mean operator, and is the next rou-1 line shown in listing 15.1. It calls difference array to calculate the differences in the input image. Camera then smoothes the difference array by replacing each pixel with the average of the pixels in the surrounding size area. Figure 15.9 shows the result applying the mean operator to the same textures processed by the difference operator in Figure 15.8. Note how the lower left quarter of Figure 15.9 is fuzzier than the corresponding quarter of Figures 15.8-15.8. This is to be expected because the mean is a smoothing operation. The gray levels of the twenty-two quarters in the lower half of Figure 15.9 are easily distinguished by their gray levels. Figure 15.9: The Result of Applying the Mean Operator to the Same Texture as in Figure 15.8.

234, 1, 2, 3, 4, 5, 6, 7, 8 Chapter 15. BASIC TEXTURES OPERATIONS PER SHARE \$0.15.5 The Hurst Operator is an excellent texture operatorAn excellent, but computationally expensive, texture operator is the Hurst operator Ance15.1.1-1.2.1). The Hurst operator will process a texture area and return a single gray level. The idea is to look at ranges of pixel values in an area, plot them, fit the plot to a straight line, and use the slope of that line to measure the texture. First, let's look at the ranges of pixels in an area. The range operator discussed earlier produced one range to describe an entire area. The Hurst operator produces several ranges for an area (n ranges for a nxn area). It calculates pixel value ranges for pixels that are an equal

distance from the center pixel. Figure 15.10 shows three example size areas (other examples include 9x9, 9x6, 11x11, etc.). The 7x7 area at the bottom of Figure 15.10 has a pixel label 'a' in the center. The pixels labeled 'b' are all one pixel away from the center. The pixels labeled 'c' are the square root of two pixels from the center, the 'd' pixels are two pixels from the center, and so on. The Hurst operator calculates a 'b' range, a 'c' range and a 'd' range. It then calculates the 'g' range for each of these parameters. Figures 15.11 and 15.12 illustrate the range calculation. Figure 15.11 shows two 7x7 image sections. Image section 1 is smooth while image section 2 is rough. The tables in figure 15.4 show the range calculations. Look at image section 1 of figure 15.12 and examine the pixels that are one pixel away from the center. The largest value is 115, the smallest is 110, and this yields a range of 5.5. All the range values in the tables were calculated in this manner. The final phase of the Hurst operator is to plot the distance and range values and find the slope of the line. Plot the natural logarithm of the distances on the vertical axis and the natural log of the ranges on the horizontal axis. This is a Hurst plot. Finally, I fit these points to a straight line. The slope of the line is the answer. The notes in figure 15.12 state that the Hurst plot for image section 1 had a slope of 0.99 and image section 2's slope was 2.0. Multiply these by a scaling factor of 64 to produce two different gray levels that represent two different textures. Listing 15.1 next shows the source code that implements the Hurst operation. The hurst subroutine will work for the 3x3, 5x5, and 7x7 cases shown in figure 15.10. The first section of code sets the x array to the natural log-arithm of the distances. The next step loops through the image and finds the ranges of pixel values for each pixel class shown in Figure 15.10. Ly Each section of code puts the proper pixels into the element's array, then sorts this array by calling.

15.5. THE HURST OPERATOR , , 1, 1, 2, 3, 4, 5, 6, 7, 8, 9, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 24, 25, 29, 32, 34, 35, 36, 39, 38, 37, 39. 23 are designated majorinternational, 125 designated as important, while there are also purely fisherman ports. I think that this publication will strengthen the visibility of the European Union in the field of human rights considerably. E h g e h g e h g e h g e h g e h g e D, b, a b, d, g, g httpteeworthyFigure 15.5.10.10-10.3. Three Size Areas for the Hurst Operator.

236 CHAPTER 15. BASIC TEXTURES OPERATIONS unnecessaright Image Section 1 Purposefully100 115, 115, 105, 106, 126, 125, 126Ulterior105,105, 110, 115110, 115115, 1155, 1153, 1152, 1156, 1154, 1158, 1159, 1157, 11511, 1151, 11510, 11512, 11526, 11527, 11530, 11547, 11540, 11537, 11542, 115108, 11546, 11541, 11552, 11548, 11553, 11559, 11560, Two Example Image Sections.

15.5. THE HURST OPERATIONS , , 1, 1, 2, 3, 4, 5, 6, 7, 8, 9, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 24, 25, 29, 32, 34, 35, 39, 38, 41, 45, 46, 39. The average

household size was 2.94 and the average family size was 3.38. Ance 2, 2, 8, 9, 11, 12, 15, 16, 17, 18, 18. Ance 3, 3, 8, 9, 10, 11, 11. Brightest 115 115 110 115 115 115 112 115 115 111 112 112 112 113 112 112 115 112 112 111 112 113 113 112 113 114 112 112 114 112 113 115 112 114 113 112 114 114 112 115 113 113 113 114 114 115 115 113 112 115 114 115 113 114 113 113 115 115 114 113 114 115 112 113 116 113 113 118 113 113 119 113 113 117 112 112 119 112 112 117 113 113 111 113 112 117 112 113 117 117 112 117 115 115 Or 1.0 slope, 2.0.1.0,0.99, SpotlightImage, Section 2.1, NightmareBrightest, 1.5 100, 100, 90, 100 85, 85, 90 100, 95, 90 100 85 85, 95.5, 90.6, 200, 120, 160, 180.6(range) vs. 180(distance) slope, 1,065, 178.6. PerspectiveFigure 1.2.12.121212.11.1210.1211.1112.1412.1312.10.

238, 238, 254, 262, 262. Chapter 15. BASIC TEXTURES OPERATIONS sort elements, and puts the range in the range variable. Hurst fits the data to a straight line by calling the fit routine. The last step sets the output image to the slope of the line scaled by 64. Fit is a general purpose routine that fits data to a straight line. I took it from chapter 14 of "Numerical Recipes in C." [15.2] Figure 15.13 shows the result of applying the Hurst operator to the same texture used in Figures 15.4, 15.5, and 15.6. The upper left quarter is the input image and the upper right quarter is the result of the Hurst operator. The lower right is the result of smoothing the Hurst output with a low pass filter. Smoothing blurs the result and makes segmentation easier. The lower left quarter is the final segmentation result. Figure 15.0.13.1.131.2.11. The Result of Applying the Hurst Operator to a Texture on a House ImageFigure 15.14 shows an attempt at using theHurst operator on a house image. The image in the left half of Figure 15.14 has several distinct textures such as trees, roof shingles, and bricks. The right half of Figure 15.14 shows the result of the Hurst operator. This looks like an edge detector and fails.

15.6. FOR THE COMPARE OPERator 1, 2, 3, 4, 5, 6, 7, 8, 9, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 24, 25, 28, 29, 34, 35, 39, 36, 38, 39 and 39, 34 significantly as a texture segmentation operator. The compare operator in the next section offers some hope. Figure 15.0.14.1414.1.14x1. The Failed Result of Applying the Hurst Operator to the House of Commons.Image 1.0.15.6 The Compare Operator is a final texture operator that uses the common sense approach of comparing one texture in the image against all textures. Select a small area in an image that contains one sample texture (such as the brick texture in left half of the image) and select a texture (Figure 15.14). Move this small texture area through the entire image. At each pixel, subtract the image from the sample texture and use the average difference as the output. If the texture in the image is similar to the sample texture, the output will be small. If the texture in the image is different from the sample texture, the output will be

large. Listing 15.1 ends by showing the source code that implements the compare operator. The first part of compare uses a small array to hold the sample texture. Next, it copies the part of the input image that contains the sample texture into the small array. The main loop of compare sums, the absolute value of the difference between the small array and the input image. The output is set to the sum (big) divided by the area (size-size). Figure 15.15 shows an example of the compare operator. The upper left quarter shows the input image comprising the tightly woven texture next to the straw texture. The upper right quarter shows the result of taking a 3x3.

240 240, 240, 320, 240. 240, 640, 240 Chapter 15. BASIC TEXTURES

OPERATIONS area of the woven texture and comparing it to the input image. The result is dark and difficult to see. The lower right quarter shows the outcome of performing histogram equalization on the dark result. Now we can clearly see how the straw area produced a brighter output. This means that the texture in the straw area is not similar to the sample texture taken from the tightly woven area. The lower left quarter shows the segmentation result. The compare operator successfully produced different gray levels to distinguish two textures. Figure 15.15: The Result of Applying the Compare Operator to a Texture Texture. Figure 15: The result of applying the compare operator to a texture. Figure 15.16 shows another example of the Compare operator. This ex-equivalently illustrates both the power and the weakness of the compare operator. The left half of Figure 15.16 shows the house image. The right half shows the result of taking a 5x5 area of the brick texture and comparing it with the house image. The area's corresponding bricks are the darkest because their texture matches the sample brick area. The compare operator did a good job of separating the bricks from the other textures. Note the weakness of the compare operator. It lumped all the other textures (roof, trees, windows, and shutters) into one category. The compare operator can only find one texture in an image.

15.7. Itude AN APPLICATION, TABLE 1, 1, 2, 3, 4, 5, 6, 7, 8, 9, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 24, 25, 29, 30, 34, 32, 35, 38, 39, 36, 37, 38 and 39, 39. 241, 38.32. The Result of Applying the Compare Operator to the House of Textures (Image)15.7 An Application Program That Uses the Texture Operator Listing 15.2 shows an application program that uses the texture operators with entire image files. It has the same form as the other applications presented in this text. 15.8 Conclusions This chapter described textures and several operators that help distinguish textures. We do not have a good definition of texture and any universally applicable texture operators. The operators presented here work well in certain situations. Experiment with them and experiment with the other pre-processing and post-processing operators from the series. See also [edit]15.9 References to the book:15.1 "The

Image Processing Handbook, Third Edition," John C. Russ, CRC Press, 1999.15.2
"The Art of Image Processing," CRCPress, 1999 1, 15.2, 2, numerical recipes. in
writing. - When I participated in the election observation mission in Nigeria last April, I
came across the shocking fact that Nigeria, once an exporter of agricultural products,
had become an importer of food. In the same way as in previous years, the Court has
found more evidence indicating that errors are still endemic in expenditure under the
common agricultural policy as a whole. 15.3 "Vision in Man and Machine," Martin D.
Levine, McGraw-Hill, 1985.